

Curso Completo: Desarrollo Backend con REST, JavaScript y Express

Duración sugerida: **12 clases (2 a 3 horas por sesión)**

Modalidad: Teórico – práctica

Proyecto final: **API REST completa con Express**

Objetivo General

Desarrollar aplicaciones backend utilizando JavaScript, Node.js y Express para crear APIs REST funcionales, organizadas y listas para conectarse con aplicaciones frontend o móviles.

Requisitos Previos

- Conocimientos básicos de programación
 - Conocimientos básicos de JavaScript
 - Editor recomendado: [Visual Studio Code](#)
 - Runtime: [Node.js](#)
 - Cliente API: [Postman](#)
-

Metodología

- Explicación teórica
 - Ejercicios prácticos
 - Desarrollo incremental de API
 - Taller por clase
 - Proyecto final
-

Competencias a Desarrollar

Al finalizar el curso el estudiante podrá:

- Comprender el funcionamiento de APIs REST
 - Crear servidores con Express
 - Manejar rutas HTTP
 - Procesar datos JSON
 - Implementar CRUD completos
 - Conectar APIs con bases de datos
 - Manejar middlewares y errores
 - Organizar proyectos backend profesionalmente
-

Estructura del Curso

CLASE 1 — Introducción al Backend, Node.js y APIs REST

Objetivos

- Comprender cómo funciona el backend
- Entender qué es una API REST

Temas

- Frontend vs Backend
- Cliente – servidor
- ¿Qué es una API?
- REST y JSON
- Instalación de Node.js
- npm y paquetes

Instalación

node -v

npm -v

Taller

- Instalar Node.js
- Crear primer proyecto Node

CLASE 2 — Introducción a JavaScript Backend

Objetivos

- Reforzar fundamentos JavaScript aplicados al backend

Temas

- Variables
- Funciones
- Arrow functions
- Arrays
- Objetos
- `map()`, `filter()`, `find()`

Ejemplo

```
const usuarios = [  
  {nombre:"Juan", edad:20},  
  {nombre:"Ana", edad:17}  
];
```

```
const mayores = usuarios.filter(u => u.edad >= 18);
```

```
console.log(mayores);
```

Taller

Crear funciones para:

- Filtrar usuarios
- Buscar productos
- Calcular totales

CLASE 3 — Primer Servidor con Express

Objetivos

- Crear un servidor web básico

Temas

- ¿Qué es Express?
- Instalación
- app.listen()
- Rutas GET

Instalación

```
npm init -y  
npm install express
```

Ejemplo

```
const express = require('express');
```

```
const app = express();
```

```
app.get('/', (req,res)=>{  
  res.send('Hola Mundo');  
});
```

```
app.listen(3000, ()=>{  
  console.log('Servidor activo');  
});
```

Taller

Crear servidor con:

- Ruta inicio
- Ruta usuarios
- Ruta productos

CLASE 4 — Métodos HTTP y REST

Objetivos

- Comprender operaciones CRUD

Temas

- GET
- POST
- PUT
- DELETE
- PATCH
- RESTful APIs

Ejemplo CRUD

```
app.post('/usuarios',(req,res)=>{  
  res.send('Usuario creado');x|  
});
```

Taller

Crear CRUD básico de estudiantes

CLASE 5 — JSON y Request Body

Objetivos

- Procesar datos enviados desde cliente

Temas

- JSON
- req.body
- express.json()
- Parámetros

Ejemplo

```
app.use(express.json());
```

```
app.post('/usuarios',(req,res)=>{
```

```
  console.log(req.body);
```

```
res.json(req.body);  
  
});
```

Taller

Crear API para registrar productos

CLASE 6 — Parámetros y Query Strings

Objetivos

- Capturar información desde URL

Temas

- req.params
- req.query
- Búsquedas y filtros

Ejemplo

```
app.get('/usuarios/:id',(req,res)=>{  
  
  res.send(req.params.id);  
  
});
```

Taller

API de búsqueda de usuarios

CLASE 7 — Middlewares en Express

Objetivos

- Comprender el flujo de peticiones

Temas

- ¿Qué es middleware?
- next()
- Logs
- Validaciones

Ejemplo

```
app.use((req,res,next)=>{  
  
  console.log('Middleware ejecutado');  
  
  next();  
  
});
```

Taller

Crear middleware de validación

CLASE 8 — Estructura Profesional de Proyecto

Objetivos

- Organizar aplicaciones backend

Temas

- Carpeta routes
- controllers
- services
- models
- Modularización

Estructura

```
src/
├── routes
├── controllers
├── services
├── models
└── app.js
```

Taller

Reestructurar proyecto CRUD

CLASE 9 — Conexión a Base de Datos

Objetivos

- Conectar Express con base de datos

Temas

- Introducción a bases de datos
- MongoDB o MySQL
- Conexión básica

Opción MongoDB

`npm install mongoose`

Ejemplo

```
mongoose.connect('mongodb://localhost:27017/test');
```

Taller

Guardar usuarios en base de datos

CLASE 10 — Manejo de Errores y Validaciones

Objetivos

- Crear APIs robustas

Temas

- try/catch
- status codes
- Validaciones
- Manejo de errores globales

Ejemplo

```
try{  
  
  res.status(200).json(data);  
  
}catch(error){  
  
  res.status(500).json(error);  
  
}
```

Taller

Agregar validaciones al CRUD

CLASE 11 — Autenticación Básica y Proyecto Final

Objetivos

- Introducir seguridad básica

Temas

- Login básico
- JWT (introducción)
- Protección de rutas

Instalación

`npm install jsonwebtoken bcrypt`

Taller

Crear login básico

CLASE 12 — Proyecto Final y Despliegue

Objetivos

- Finalizar y presentar API REST

Proyecto Final

Crear una API REST completa para:

- Usuarios
- Productos
- Categorías
- Login

Debe incluir

- CRUD completo
- Middlewares
- Validaciones
- Base de datos
- Organización profesional

Despliegue

Introducción a:

- VPS
 - Render
 - Railway
-

Proyecto Final Sugerido

Sistema de Inventario API REST

Módulos

- Usuarios
- Productos
- Categorías
- Autenticación

Endpoints

GET /productos

POST /productos

PUT /productos/:id

DELETE /productos/:id

Evaluación

Item	Porcentaje
Talleres	30%
Participación	20%
Proyecto Final	50%

Herramientas Recomendadas

Backend

- [Node.js](#)
- [Express.js](#)

Testing APIs

- [Postman](#)
- [Insomnia](#)

Base de Datos

- [MongoDB](#)
 - [MySQL](#)
-

Recursos de Apoyo

Documentación Oficial

- [MDN JavaScript](#)
- [Express Documentation](#)
- [Node.js Docs](#)